



UNIVERSITÀ
DI TRENTO

Delivery Autonomus Agents

Autonomous Software Agents

a.y. 2025/2026

Davide Donà - 264467

davide.dona-1@studenti.unitn.it

Andrea Blushi - 264468

andrea.blushi@studenti.unitn.it

June 2026

Contents

1 Problem Statement	1
2 BDI Agent	1
2.1 Belief Revision	2
2.1.1 Agent beliefs	2
2.1.2 Map beliefs	2
2.1.3 Parcel beliefs	3
2.1.4 Crate beliefs	3
2.2 Desires and Intention Selection	3
2.2.1 Queue reconstruction	3
2.2.2 Dynamic Scoring	3
2.3 Planning: A* Navigation and Collision Handling	5
3 PDDL-Based Crate Navigation	6
3.1 Domain Overview	6
3.2 Planning With PDDL	7
3.3 A Smart Solution: Problem Restatement & Caching	7
4 LLM-Based Agent Layer	8
4.1 Adapting Behavior Through Goal Injection	8
4.2 Adapting Strategy via Rule Injection	9
4.3 Coordination and Communication	10

1 Problem Statement

The `Deliveroo.js` environment is a grid-based delivery simulation where autonomous agents collect and deliver scattered parcels to designated tiles to maximise their accumulated score. The environment introduces various challenges: agents only perceive tiles within a limited sensor radius at any given time step. Parcel rewards decay at a fixed server-configured interval, creating time pressure that incentivises early collection. Multiple agent types may share the map; agents belonging to the same team cooperate to jointly maximise reward, whereas agents from different teams compete for the same parcels. Additionally, movable crates often obstruct navigable paths, meaning agents must actively push them to clear blocked routes.

We implement a BDI (Belief-Desire-Intention) agent augmented with an optional Large Language Model (LLM) based control layer. The BDI core handles reactive parcel collection, exploration, and crate-clearing autonomously; the LLM layer intercepts high-level instructions, adapts agent behaviour through a set of structured tools, and coordinates multi-agent strategy via periodic team assignments.

2 BDI Agent

The agent operates on a continuous cycle driven by the `sensing` socket event emitted by the server. Rather than relying on a static script, the agent dynamically reacts to the environment through a strict, four-stage pipeline. Upon receiving a `sensing` event, the following sequence executes:

1. **Perception** (`perceive()`): The agent integrates the new sensory data into its belief state through a Belief Revision process (detailed in Section 2.1). The actions we perform that modify the belief state automatically updates the beliefs, without having to wait for the next sensing event.
2. **Deliberation** (`deliberate()`): Using its updated beliefs, the agent generates potential goals (desires) and calculates a score for each. It then rebuilds its intention queue, prioritizing the most valuable task (detailed in Section 2.2).
3. **Planning** (`Planner.plan()`): The planner extracts the highest-ranked intention from the queue and translates it into a concrete sequence of executable, typed steps (detailed in Section 2.3).
4. **Execution** (`Executor`): Finally, the planned actions are asynchronously emitted to the server via socket events. To maintain responsiveness, the executor triggers an action at each server clock tick, instead of waiting for a perception update or an acknowledgment from the server.

2.1 Belief Revision

The agent’s belief state relies on two foundational utility data structures to manage temporal observations, which in turn feed into three specialized sub-systems: **AgentBeliefs**, **MapBeliefs**, **ParcelBeliefs**.

Tracker **Tracker** stores a single latest observation $\langle v, t_{seen} \rangle$ per entity **e**. On each sensing report, **update(e, v)** overwrites the existing entry. The confidence of a tracked observation decays exponentially over time:

$$c(t) = \exp\left(-\frac{t - t_{seen}}{\tau}\right) \quad (1)$$

where τ is the exponential time constant (the duration after which confidence drops significantly).

When the current sensing window covers an entity’s last-known position, but the entity is absent from the report, **invalidateAtSensedPositions** nullifies that position without removing the record entirely.

Memory While the tracker maintains only the latest state, **Memory** is a supplementary store that maintains a bounded temporal history of observations per entity, automatically evicting entries older than a defined time threshold.

2.1.1 Agent beliefs

AgentBeliefs stores the latest state of every observed agent via **Tracker**, and a bounded position history for each enemy via **Memory**. It also maintains a spatial heat field over the map, aggregating enemy presence across all tracked observations.

2.1.2 Map beliefs

MapBeliefs stores the static tile layout, a set of transient blocked tiles each with a configurable TTL, and soft traversal penalties on individual tiles.

Upon map receipt, it identifies and permanently seals *sink tiles*: tiles that an agent can enter but from which no valid pickup-and-deliver loop exists. This situation arises on maps where directional conveyor tiles create one-way corridors: an agent may be carried into a region from which it can never return to a spawn tile, or from which no delivery tile is reachable.

To detect these dead zones, the map is modelled as a directed walkability graph where conveyors enforce one-way edges. Tarjan’s algorithm is then applied to find all Strongly Connected Components (SCCs). An SCC is considered *safe* only if it contains at least one spawn tile *and* at least one delivery tile — i.e. it supports the full pickup-and-deliver cycle. All walkable tiles outside a safe SCC are reclassified as walls, preventing the planner from ever routing the agent into an unrecoverable region.

2.1.3 Parcel beliefs

This component tracks the availability and value of delivery targets. Parcels within the current sensing window receive their authoritative reward from the server. Once out of sight, parcel rewards decay linearly at -1 per server-defined interval Δ_t . Each parcel maintains an independent decay clock to prevent drift accumulation. Given a parcel last observed at time t_0 with reward r_0 , its estimated reward at time t is:

$$r(t) = r_0 - \left\lfloor \frac{t - t_0}{\Delta_t} \right\rfloor. \quad (2)$$

When $r(t) \leq 0$, the parcel is considered expired and is deleted from the agent's beliefs.

2.1.4 Crate beliefs

`CrateBeliefs` uses the same `Tracker` utility described above to store the latest known position for each crate. On every sensing update, observed crate positions are registered and `invalidateAtSensedPositions` nullifies any entry whose tile falls within the current sensing window but is absent from the report, reflecting that the crate has been pushed.

2.2 Desires and Intention Selection

For the BDI layer, we define a fixed set of 3 core desire types, each parameterized by a target tile:

- **ReachParcel**: navigate to and collect a visible parcel;
- **DeliverParcel**: navigate to a delivery tile while carrying parcels;
- **Explore**: navigate to an unobserved spawn tile when idle (i.e., no reachable parcels or deliveries).

2.2.1 Queue reconstruction

At each deliberation step, the intention queue is fully reconstructed from the current belief state. The queue is sorted by priority first, then by descending score within each desire type. Both `ReachParcel` and `DeliverParcel` desires have a priority of 1, while `Explore` has a lower priority of 0. This ensures that the agent always prefers tangible parcel-related goals over exploration when any are available.

2.2.2 Dynamic Scoring

The agent computes a dynamic score for each desire to guide intention selection. The score is a real-valued estimate of the desire's expected utility, incorporating factors such as the parcel's reward, the distance to the target, and the presence of competitors.

Distance Penalty Each desire’s score is inversely proportional to the estimated distance to its target, calculated using the agent’s current beliefs. The agent computes the shortest path distance d from its current position to all other tiles using a Breadth-First Search (BFS). We will refer to this distance as d throughout the rest of the section. (Note: when $d = 0$, we assign an infinite score).

ReachParcel Let r_p denote the current estimated reward of a target parcel p , and C the set of parcels the agent is currently carrying. The score S for a **ReachParcel** desire targeting parcel p is calculated as:

$$S = \frac{r_p + \sum_{c \in C} r_c}{d} \cdot \gamma$$

Where γ is a race factor that de-prioritizes parcels likely to be collected by competitors. It is computed as:

$$\gamma = \min \left(1, \frac{d_{\text{enemy}}}{d} \right)$$

where d_{enemy} is the distance from the nearest tracked enemy to the parcel.

Incorporating the total reward of currently carried parcels ($\sum_{c \in C} r_c$) into the score allows the agent to evaluate the utility of picking up a new parcel against the potential value of delivering existing ones. This effectively balances the agent’s acquisition and delivery goals.

DeliverParcel For a **DeliverParcel** desire, the objective is to secure the rewards of the parcels currently held by the agent. The score S is computed as the total estimated reward of all carried parcels C , inversely scaled by the shortest path distance d to the delivery tile:

$$S = \frac{\sum_{c \in C} r_c}{d}$$

Explore The **Explore** desire serves as a fallback to locate newly spawned parcels. We first define the normalized observation age τ :

$$\tau = \min \left(1, \frac{\Delta t}{T_{\text{spawn}}} \right)$$

where Δt is the time since the target spawn tile x was last observed, and T_{spawn} is the environment’s spawn interval.

The cluster weight w quantifies the structural density of the target’s surrounding spawn area. It is computed as the sum of inverse distances to all neighbor spawn tiles within the observation range R_{obs} :

$$w = \sum_{y \in OST} \frac{1}{\delta(x, y) + 1}$$

where OST is the set of spawn tiles such that their Manhattan distance $\delta(x, y) \leq R_{\text{obs}}$.

The enemy heat h actively steers the agent away from sectors currently crowded by competitors. It is accumulated from the history of tracked enemy observations E , combining spatial and temporal decay:

$$h = \sum_{e \in E} \exp\left(-\frac{d_e^2}{2\sigma^2}\right) \exp\left(-\frac{\Delta t_e}{\tau_h}\right)$$

where d_e is the spatial distance to the past enemy observation, Δt_e is the observation's age, σ controls the spatial spread, and τ_h is the temporal decay rate.

Finally, the score for an **Explore** desire is computed as:

$$S = \frac{w \cdot \tau}{d \cdot (1 + h)}$$

2.3 Planning: A* Navigation and Collision Handling

For pathfinding, the agent employs a standard A* search algorithm with a Manhattan distance heuristic, which is admissible in our grid-based environment. The planner incorporates dynamic walkability checks based on the agent's current beliefs, treating temporarily blocked tiles as impassable. Statically, walls and out-of-bounds tiles are always rejected; conveyor tiles are additionally direction-sensitive, blocking movement against their flow.

Plans generated by the A* algorithm are defined as sequences of typed actions: directional **move** steps followed by a terminal **pickup** or **putdown** step. An existing plan is reused across deliberation steps as long as the active desire type and target remain unchanged, and all remaining steps successfully pass walkability re-validation.

Collision Handling Prior to executing a move step, the agent verifies the availability of the target tile. A tile is considered occupied, and thus temporarily blocked, under two conditions: either an enemy is currently observed on it, or an enemy's predicted next position matches it with sufficient confidence.

When the next planned tile is occupied, the collision manager applies a three-stage escalation:

1. **Detour**: replan with the blocked tile treated as a wall. This is accepted only if the detour adds at most a predefined threshold of Δ_{th} steps over the remaining plan.
2. **Wait**: if no acceptable detour exists, pause execution for a random duration drawn from a predefined uniform distribution.

3. **Block and replan:** after the wait expires, or after a specified limit L of repeated invalidations, mark the tile as temporarily blocked for a randomized time-to-live (TTL) duration and completely replan toward the same goal.

3 PDDL-Based Crate Navigation

Some maps in the Deliveroo.js environment contain movable crates that can obstruct navigable paths. An agent may push a crate one tile in any cardinal direction, subject to the constraints outlined in Section 2.3. We model this crate manipulation problem using PDDL (Planning Domain Definition Language), invoking it as a fallback mechanism when the standard A* planner fails to find an unobstructed route.

3.1 Domain Overview

The `deliveroo-crates` domain is formulated in STRIPS and relies on two primary object types: `tile` and `crate`. The complete state required for crate-aware navigation is captured by the following predicate set:

- `(at ?t)`: The agent is located on tile t .
- `(adj-up/down/left/right ?from ?to)`: Defines the valid directed spatial relationships between adjacent tiles for movement or pushing.
- `(crate-at ?c ?t)`: Crate c occupies tile t .
- `(crate-free ?t)`: Tile t contains no crate, making it available for entry or as a push destination.
- `(crate-space ?t)`: Tile t can structurally support a crate (e.g., it is not a wall or a conveyor belt).

The domain defines eight parameterized actions: four directional *move* actions and four directional *push* actions.

- A move action allows the agent to step into an adjacent `crate-free` tile.
- A push action allows the agent to displace a crate one tile in the specified direction, requiring two preconditions:
 - (i) the agent is positioned adjacent to the crate on the opposite side of the push direction, and
 - (ii) the target destination is a valid `crate-space` and is currently `crate-free`.

The execution of a push action simultaneously relocates the crate to the target tile and moves the agent onto the crate's former tile, updating the `crate-free` state of the involved tiles accordingly.

3.2 Planning With PDDL

PDDL planning is invoked exclusively when standard A* pathfinding fails and the agent’s belief system indicates that crates obstruct the route (Section 2.3). This detection occurs in two phases. First, an A* search is executed, treating all crate-occupied tiles as impassable. If this search fails, a second A* search is performed that entirely ignores crate positions, yielding a structurally valid but potentially obstructed path. Any crates from the agent’s current beliefs that intersect this path are collected; their presence confirms that the route is blocked by movable obstacles rather than static map geometry.

A PDDL problem instance is subsequently constructed from the agent’s current beliefs. The `:init` state is populated with the following:

- `(at t_X_Y)`, where (X, Y) is the agent’s current position;
- adjacency facts for every pair of connected non-wall tiles, covering all four cardinal directions;
- `(crate-at ?c ?t)` and `(crate-free ?t)` facts for each believed crate position;
- `(crate-space ?t)` facts for all structurally valid positions (non-wall, non-conveyor tiles).

The `:goal` section comprises a single ground atom, `(at t_TX_TY)`, where (TX, TY) represents the target position. Consequently, the solver is tasked with generating a *complete* sequence of `move` and `push` actions from the agent’s current location to the final destination, clearing any obstructing crates along the way. The resulting plan is translated back into the agent’s internal action representation, stored in the plan buffer, and executed step-by-step until completion or failure.

However, this formulation suffers from a fundamental performance bottleneck: the PDDL solver is slow for the interactive demands of the environment. An initial optimization involved transitioning to a local solver, which eliminated the overhead of remote API requests and constant problem serialization. A more structural optimization, caching solved plans, is unfortunately undermined by the formulation itself. Because the cache key must incorporate both the start and goal tiles, even a trivial change in the agent’s position alters the key and triggers a cache miss, despite the crate-clearing maneuver remaining identical regardless of where the agent initiated the request.

3.3 A Smart Solution: Problem Restatement & Caching

The root cause of this inefficiency is that PDDL is only strictly necessary for navigating the crate cluster itself; the initial and final segments can be efficiently handled by standard A* navigation.

To resolve this, the problem is reformulated so that the PDDL solver is only responsible for the segment of the journey that requires crate manipulation. The overall plan is decomposed into three distinct segments:

- Move from the current position to the cluster entry tile (A^*);
- Navigate through the crate cluster, from entry to exit (PDDL);
- Move from the cluster exit tile to the final goal (A^*).

Under this paradigm, the PDDL problem is anchored to the cluster’s entry tile rather than the agent’s live position. Computed maneuvers are stored in a FIFO cache, utilizing the tuple (entry, exit, {crate positions}) as the key.

4 LLM-Based Agent Layer

The LLM layer functions as a thin wrapper around the underlying BDI agent. It interprets high-level textual commands from a mission controller, utilizing tools exposed by the BDI core to inject new desires, adjust scoring rules, or initiate team coordination.

The mission controller can issue requests that fall into three broad categories: direct navigation goals, persistent behavioural adjustments, and team-level control. Each category is handled through a different mechanism in the BDI core, chosen to integrate naturally with the existing deliberation loop rather than bypassing it.

4.1 Adapting Behavior Through Goal Injection

The simplest requests are navigation goals: move to a tile, or deliver at a specific location. These are translated into desires that enter the BDI intention queue and compete with existing goals through the standard scoring mechanism. A *go-to* request creates a **REACH_TILE** desire with a configurable reward and expiry time; the reward determines how aggressively the desire competes against parcel collection. A *deliver-at* request creates a **DELIVER_PARCEL** desire targeting a specific tile, with an optional bonus reward that biases the delivery score for that destination (Section 4.2).

All those requests have a limited duration to be executed. The generated desire are made persistent for the duration of the specified TTL, so that they can be continuously re-evaluated and pursued until completion or expiry.

Both desire types use the same rate formula as their BDI counterparts (Section 2.2). A **REACH_TILE** desire is scored as:

$$S = \frac{r_{\text{inj}} + \sum_{c \in C} r_c}{d}$$

where r_{inj} is the configurable reward supplied by the mission controller and $\sum_{c \in C} r_c$ is the total estimated value of currently carried parcels. This mirrors the **ReachParcel** formula, so a navigation goal with a reward equal to a typical parcel value competes on equal footing with parcel collection. A **DELIVER_PARCEL** desire applies the same rule-adjusted scoring as the BDI deliv-

ery desire, with an optional additive bonus applied after the rule combination, allowing the mission controller to bias delivery toward a specific tile.

4.2 Adapting Strategy via Rule Injection

The **RuleStore** maintains a persistent registry of LLM-defined scoring rules. Each rule encodes a state condition alongside a linear effect: a multiplier m and an additive offset a .

During each deliberation cycle, any matching rules are applied to the baseline scores from Section 2.2 as a linear combination. This scales or shifts the priorities without overwriting the foundational scoring logic.

These rules evaluate three primary state variables: the individual parcel reward, the current carry count, and the target delivery tile. Their effects compose directly onto the reward aggregates utilized in the desire formulas.

ReachParcel: The baseline score (Section 2.2) is linearly adjusted according to the *estimated stack count* post-pickup: the hypothetical carry count $|C| + 1$. This allows a rule to incentivize picking up a specific parcel only if it achieves a desired total stack size, leaving all other pickup valuations unchanged.

DeliverParcel: The delivery score is linearly modified based on the current carry count $|C|$ and, optionally, the specific target tile. A stack rule keyed to $|C|$ dynamically alters the attractiveness of delivery depending on the agent’s current payload, while a tile-specific rule can shepherd the agent toward a preferred drop-off location. An independent bonus can also be applied directly via the `request_putdown_at` tool, biasing delivery toward a specific tile regardless of any active rules.

Explore: In the absence of an active stack rule, **Explore** maintains its default priority of 0 and functions as outlined in Section 2.2.

However, when a positive stack rule is triggered, **Explore** is elevated to priority 1, forcing it to compete directly against active parcel and delivery goals. Its score is subsequently scaled by the anticipated reward of completing the stack, incentivizing the agent to actively search for missing parcels rather than settling for a premature delivery. This behavior emerges naturally from the linear weighting system: the LLM is not required to issue an explicit exploration command, as simply registering a stack rule is sufficient to trigger the shift.

Traversal Penalties: Traversal penalties operate at the planning layer rather than the deliberation layer. Unlike scoring rules, which modulate desire priority during deliberation, a traversal penalty injects an additive cost onto the A* edge leading onto a specific tile. When the planner evaluates candidate paths, it adds this cost to any step landing on a penalized tile, causing the agent to

prefer a detour whenever the detour length is shorter than the penalty, without completely forbidding the option of traversing the tile if it becomes necessary.

4.3 Coordination and Communication

Peer Communication: Simple team-level commands (navigation goals, scoring rules, traversal penalties) follow a uniform two-step pattern: the injection is applied to the issuing agent locally, then packaged as a peer-injection message and broadcast to all teammates. Commands that carry stronger team-synchronisation requirements — rendezvous and traffic control — instead use a two-phase commit protocol described below.

Beacon Communication: To handle the critical need for up-to-date position information in coordination maneuvers, each agent continuously broadcasts its current position to all configured teammates as a `position_beacon` message at a fixed interval. This ensures that every team member maintains an up-to-date position estimate for its peers regardless of the game’s sensing radius, which only reveals agents within a limited Manhattan distance.

Rendezvous and Traffic Control: Both `request_rendezvous` and `request_red_light` employ a two-phase commit (2PC) protocol before any agent holds, ensuring the team only stops when it is collectively optimal to do so.

In Phase 1, the proposing agent broadcasts a *propose* message carrying a unique round identifier (`rid`) and the command parameters. Each receiving teammate independently evaluates whether accepting is beneficial: it computes the best hold-tile score achievable under the proposal and compares it against its current best `REACH_PARCEL` or `DELIVER_PARCEL` score. For rendezvous this uses the joint-bottleneck formula (Section 2.2); for traffic control it uses the standard hold-tile rate score. The teammate replies with a *vote* message containing the `rid` and its accept/reject decision.

In Phase 2, after a fixed collection window of 750 ms, the proposer checks for unanimity. If every expected teammate voted *accept* for the current `rid`, the proposer broadcasts a *commit* message; each agent (including the proposer) then injects the `HOLD_TILE` desires into its intention pool. If any vote is missing or negative, the proposer broadcasts *abort* and no holds are injected on any agent.

Upon commit, rendezvous holds carry a `releaseZone` that triggers automatic release once position beacons confirm all teammates are within the designated Manhattan radius — no green-light command is needed. Traffic-control holds have no release zone and persist until their TTL expires or `request_green_light` is issued, which clears all active `HOLD_TILE` intentions on both agents immediately via a fire-and-forget broadcast. By default agents hold on the nearest odd-row tile, but the command accepts optional `axis` (row/column) and `parity`

(`odd/even`) parameters to stagger agents across any row or column alignment, enabling flexible traffic separation strategies.

Coordinator: A centralized **Coordinator** periodically executes team assignment rounds. During each round, a `request_beliefs` envelope is broadcast to all teammates. After a short collection window, the coordinator constructs a geometric summary of the team’s state (including agent positions, payloads, and high-value zones) and feeds it to `LLMClient.coordinate`. This triggers `assign_goto` commands to optimally distribute the agents. The mission controller can also trigger this process on demand via the `request_team_status` tool.